

Clinical Cluster Phenotype Card

Schema Specification and Implementation Guide

Arootin Gharibian

June 8, 2026

Abstract

This document describes the PhenoCard (the *Clinical Cluster Phenotype Card*), a schema for sharing clinical cluster profiles. The schema provides a structured representation for semi-supervised, cluster-derived patient profiles, designed to reproduce feature-subset values and human-in-the-loop feature selection, with class-matching clusters as the unit of evaluation. The cluster feature values, as computable artifacts, can be published with provenance and cluster-label matching metrics, allowing independent nodes to compare and reproduce clusters across heterogeneous datasets in an algorithm-independent, privacy-preserving manner.

This documentation is licenced under European Union Public Licence v. 1.2

Contents

1	Overview & Methodology	2
2	Information Privacy	2
2.1	Privacy Guarantees	2
3	Structure & Schema Specification	3
3.1	Metadata and Provenance Layer	3
3.2	Statistical and Distributed Mathematics Layer	3
3.3	Phenotype Signature Layer	3
3.3.1	Algorithmic Protocol (MINIMAR)	4
4	The Integrity Hash	4
5	Workflow & Implementation Guide	4
5.1	Phase 1: Producer Workflow (Generating the Card)	4
5.2	Phase 2: Consumer Workflow (Validating the Card)	4
6	Software Implementations	5
6.1	Python based JSON PhenoCard Conversion	5
6.2	Declarative UI Layout Engine and PhenoCard Viewer	5
6.2.1	The UI Layout Schema Specification	5
6.2.2	Layout View Component Hierarchy	6
6.2.3	Reference Client Runtime Architecture: The PhenoCard Viewer	6
7	Clinical Application	7
8	Appendix: JSON Schema	8

1 Overview & Methodology

The **Cluster Phenotype Card** is a schema-driven JSON file designed to standardize sharing complex clinical clusters across federated healthcare networks. By separating the mathematical data payload (Level 1-3 baseline stats and phenotype signatures) from the presentation layer (UI Schema), this repository allows for the output of FAIR-compliant profiles that can be rendered into interactive, clinician-ready dashboards.

The implementation follows a *Define–Validate–Share* pipeline designed to follow FAIR (Findable, Accessible, Interoperable, Reproducible) principles across. While standards such as PMML and ONNX excel at serializing mathematical weights for execution, they lack the clinical semantics required for semi-supervised clustering. Packaging standards such as RO-Crate provide excellent provenance support but lack domain-specific structures for clinical cluster definitions.

The PhenoCard acts as the bridge between these layers: it is a machine-readable JSON-LD document and functions as a specialized RO-Crate combining clinical semantics with a mathematical definition of the clusters. These profiles can be used to create standard RO-Crates focused on **phenotypic cluster definitions**, **statistical reproducibility**, and **computable payloads**.

The schema is built around three design principles:

1. **Reproducibility:** Enforces structured metadata for the algorithmic protocol, explicitly documenting data preprocessing, missingness handling, and the clustering methodology.
2. **Concept Interoperability:** Uses standardized concept identifiers to define the features of the cluster. The schema is vocabulary-agnostic: OMOP Athena concept IDs are the reference implementation, but nodes using other vocabularies (e.g. SNOMED CT directly, LOINC, or local codes) may supply their mapping convention separately.
3. **Privacy-Preserving Distributed Data Mining:** Embeds statistical artifacts (covariance matrices and validity scores) directly into the schema. This enables federated nodes to validate stable clusters mathematically without exposing patient-level data.

2 Information Privacy

The Cluster Profile Card is explicitly engineered for Privacy-Preserving Data Mining (PPDM). The artefact transmits only aggregate statistics and standardized concept codes; no individual patient data or clinical notes are included or permitted. Local ethics review and institutional data governance requirements apply independently of schema compliance.

2.1 Privacy Guarantees

- **No Row-Level Data:** The schema contains no fields for individual patient records. All values are population-level aggregates.
- **Aggregate Statistics Only:** All metrics (`n_count`, `dist.mean`, `covariance_value`) are computed at the cluster level.
- **Minimum Count Threshold:** The schema enforces `n_count` ≥ 10 at the cluster level and `total_study_n` ≥ 32 at the dataset level. Producers should apply the minimum threshold appropriate for their institutional and jurisdictional requirements before generating a card.

3 Structure & Schema Specification

The schema is a JSON-LD document with strict typing (`additionalProperties: false` at every level) to ensure that valid cards are computationally reproducible. It is organized into four layers described below.

3.1 Metadata and Provenance Layer

Purpose: Governance, lineage, and interoperability.

- Uses standard vocabulary mappings (`dcat:publisher`, `dcat:license`, PAV authorship predicates) for linked-data interoperability.
- Records the computational run via `provenance`: timestamps (`execution_start_time`, `execution_end_time`), optional pipeline identifier (`pipeline_run_id`), optional code version (`git_commit`), and optional dataset hash (`data_version_hash`).
- Links to the upstream cohort definition (`base_cohort_reference`) that defines the baseline study population.

3.2 Statistical and Distributed Mathematics Layer

Purpose: Distributed Computability.

- **Population Scope:** Absolute patient counts (`n_count`) and sub-population fractions relative to the profile and the full study.
- **Quality Metrics:** Internal cohesion (e.g. Silhouette score), external separation (e.g. confusion matrix components), and class label distribution. Metrics are stored as a named list under `quality_metrics.metrics`, each carrying a `metric_type` classifier.
- **Distributed Payloads:** Embeds pairwise feature covariance matrices (`covariance_matrix`) and upstream model feature importance scores (`covariate_importance`) to support federated reconstruction of cluster spaces.

3.3 Phenotype Signature Layer

Purpose: Machine-executable clinical logic.

- **Feature Set:** The defining set of clinical variables for the cluster (`phenotype_signature.features`), each mapped to a concept identifier.
- **Inclusion Criteria and Distributional Summaries:** Each feature carries both deterministic inclusion rules (e.g. `anchor_operator: ">="`, `anchor_value: 0.6`) and probabilistic distributional parameters (`dist_mean`, `dist_sd`).
- **Clustering Quality Descriptor** (`clustering_outcome`): A four-value enum describing cluster separation and label consistency: `distinct_and_pure`, `distinct_but_mixed`, `indistinct_but_pure`, `indistinct_and_mixed`.

3.3.1 Algorithmic Protocol (MINIMAR)

Purpose: Methodological reproducibility.

- Documents the algorithm configuration: method family, base model, hyperparameters, random seed, and software version (`algorithmic_config`).
- Explicitly defines the data preprocessing pipeline, including missing data strategy and feature scaling method (`data_preprocessing`).
- Maps the cluster to a human-readable clinical label (`assigned_class`) and an optional concept identifier (`assigned_concept_id`) in `label_relationship`.

4 The Integrity Hash

The optional `integrity_hash` field provides a SHA-256 fingerprint of the feature subset. When present, it allows a consuming node to verify that the cluster definition has not been altered in transit. Because JSON objects are inherently unordered, producers **must** follow a deterministic serialization protocol before hashing (typically sorting `concept_id` values and the feature array) to ensure the hash is reproducible across environments. The convention must be documented in the accompanying specification or `minimar_reference` file. Note that `integrity_hash` is optional (`Required: false`). Cards without it are valid; the field is recommended when federated integrity verification is a requirement of the receiving network.

5 Workflow & Implementation Guide

The Cluster PhenoCard is designed to be generated and consumed programmatically.

5.1 Phase 1: Producer Workflow (Generating the Card)

1. **Map Features to Concept Identifiers:** Ensure all feature variables are mapped to concept IDs within the `phenotype_signature.features` array. For OMOP networks, use ATHENA concept IDs (SNOMED CT, LOINC, RxNorm). For other networks, document the vocabulary convention in `minimar_reference`.
2. **Report Methods (MINIMAR):** Populate the `algorithmic_config` block with the exact hyperparameters, random seed, and `data_preprocessing` strategy used during the run.
3. **Calculate Metrics and Artefacts:** Compute `quality_metrics` (e.g. Silhouette score, confusion matrix components, class distribution counts) and the `covariance_matrix` on local data.
4. **Sign the Artefact (Optional):** Generate the `integrity_hash` (SHA-256) over the canonical feature set to allow downstream verification.

5.2 Phase 2: Consumer Workflow (Validating the Card)

1. **Resolve Concepts:** Read the `concept_id` and `unit_concept_id` values from the producer's feature list and map them to the local vocabulary if needed.
2. **Query Local Data:** Extract corresponding patient data from the local dataset, applying the producer's `data_preprocessing` strategy as documented in `algorithmic_config`. It is possible to filter the local population using the `dataset_definition`.

3. **Run Clustering Algorithm:** Define an algorithmic process for unsupervised or semi-supervised learning, and run the algorithm on your own data, process the clusters and calculate the values for measurement of internal and external validity. The algorithm can be either the same as source hospital or different from it.
4. **Evaluate and Compare:** Compare local population statistics against the `population` block and `anchor_value` constraints defined in `phenotype_signature.features`. If operating in a distributed network, reconstruct the local covariance matrix and compare against the producer’s `covariance_matrix`. If all checks, evaluate the external and internal validity measures to confirm or discard a cluster based on its clinical relevance.

6 Software Implementations

6.1 Python based JSON PhenoCard Conversion

The following Python scripts implement the *Define-Validate-Share* pipeline:

- **Template Generator** (`create_template.py`): Generates an standardized Excel input file, the relevant information is filled in by a user who discovered an interesting cluster.
- **Profile Builder** (`profile_builder.py`): The script reads the Excel file and outputs a validated JSON card.

6.2 Declarative UI Layout Engine and PhenoCard Viewer

To ensure complete decoupling between core data pipelines and visual representation, we use a declarative UI layout framework. The rendering layer is split into two primary components: a declarative configuration specification governed by a layout schema (`PhenoCard-UI.schema.json`) and a lightweight POC client-side runtime engine (**PhenoCard Viewer**) that accepts user input for a PhenoCard data instance and the layout schema to draw interactive HTML output.

6.2.1 The UI Layout Schema Specification

The presentation architecture is driven by a schema that defines a parameterized template tree that binds structural UI blocks to underlying JSON-LD data points via JSONPath evaluation. The layout schema contains four structural sections:

1. **Runtime Parameters:** Explicit constraints required by the engine prior to evaluation (e.g., `profile_id`). The engine enforces validation of these parameters before initializing data path evaluation.
2. **Data Contexts and Bound Data Resolution:** To eliminate repetitive syntax, the schema maps symbolic context names to explicit JSONPath expressions. The environment establishes four core scopes:
 - `@global`: Maps to global metadata blocks (`$.global_metadata`).
 - `@dataset`: Maps to study-wide population characteristics (`$.dataset_metadata`).
 - `@target_profile`: Resolves directly to the clinical profile indices: (`$.profiles['${profile_id}']`).
 - `@cluster_list`: Evaluates object properties across multi-cluster collections using dictionary wildcards (`$.profiles['${profile_id}'].clusters.*`).

3. **Runtime Computed Fields:** The framework facilitates runtime statistical aggregation over context lists. These are declared natively in the schema configuration via target expressions:
 - **sum:** Accumulates raw counts across sub-structures (e.g., total profile evaluation cohort sizes).
 - **weighted_mean:** Evaluates normalized metrics across clustered weights, such as deriving profile-wide weighted Silhouette indices by multiplying cluster scores against population sizes.
 - **group_sum:** Groups and collapses class-wise categorical metrics across split-cluster arrays.
4. **Asset Resolution Rules:** Declares binary external elements (e.g., static network diagrams, cluster-specific visual clusters) matching naming patterns such as `cluster_viz_${cluster_id}.${ext}`.
This enables dynamic rendering mapping during collection loops.

6.2.2 Layout View Component Hierarchy

The `layout` field specifies a nested component graph. The framework utilizes a series of standardized layout primitives to model the presentation space:

- **Document / PageHeader:** Structural roots governing identity blocks, titles, and clinical source attribution bindings.
- **Section / LayoutRow / LayoutColumn:** Multi-column flex grids enabling scalable viewport responses across varied screen topologies.
- **MetadataBlock / MetricGroup:** Atomic containers that isolate key-value attribute tags (e.g., missing data strategies, base model thresholds) and highlight tabular scoring dimensions.
- **DataGrid:** A structured data matrices controller designed to project composite array feeds (such as baseline demographics or clinical Table 1 equivalents) into rows and cells.
- **Iterator:** A logical looping driver that clones interior layout structures across resolved context collections (e.g., rendering structural blocks for each cluster assigned to a profile index).

6.2.3 Reference Client Runtime Architecture: The PhenoCard Viewer

Visual compilation is handled entirely inside a client-side HTML5/JavaScript application layer (**PhenoCard Viewer**). Built without external runtime frameworks, the app utilizes low-level virtual DOM patterns via a lightweight micro-element creator ($\mathcal{H}$ -factory function).

Interactive Capabilities: The viewer provides an interactive dropzone interface supporting federated, zero-server data ingestion through client-side file pickers. Once standard JSON data arrays and associated relational files (such as graphical representations) are parsed in-memory, the engine exposes a responsive, tabbed control navigation system for multi-profile clinical reviews. Network elements utilize custom bindings integrated through client visualization libraries (**Graphology** and **Sigma.js**) to coordinate localized cluster-highlighting interactions.

Parametric Boxplot Generation: Rather than calling server-side image render pipelines, the engine interprets numeric data blocks at runtime to build mathematical vector graphics on-the-fly. The custom vector assembly engine constructs SVG-wrapped parametric box-plots visualizing distribution properties directly in line with text blocks. The vector dynamically computes and overlays bounding scales, projecting standard deviation and mean distributions across visual bounds:

$$X_{\text{scaled}} = \Delta_{\text{pad}} + \left(\frac{V - V_{\min}}{V_{\max} - V_{\min}} \right) \times W_{\text{drawable}} \quad (1)$$

- **Whiskers:** Visual lines representing the outer clinical baseline bounds, positioned at $\mu \pm 2\sigma$.
- **Interquartile Analog Box:** Encapsulating $\mu \pm 1\sigma$.
- **Mean Indicator:** Statistical average (μ).

Document Printing Synthesis: Static document distribution capability is preserved using targeted CSS Media Queries (`@media print`). When print triggers execute, the DOM strips application headers, navigational components, file controls, and interactive buttons from the layout tree. Responsive layout rows are normalized to static vertical paths, and specialized CSS rule bindings (`break-inside: avoid;`) are applied globally to baseline matrix containers, SVG graphs, and metadata blocks. This isolates blocks from cross-page visual fragmentation, outputting structural PDF summary records via standard browser printing engines.

7 Clinical Application

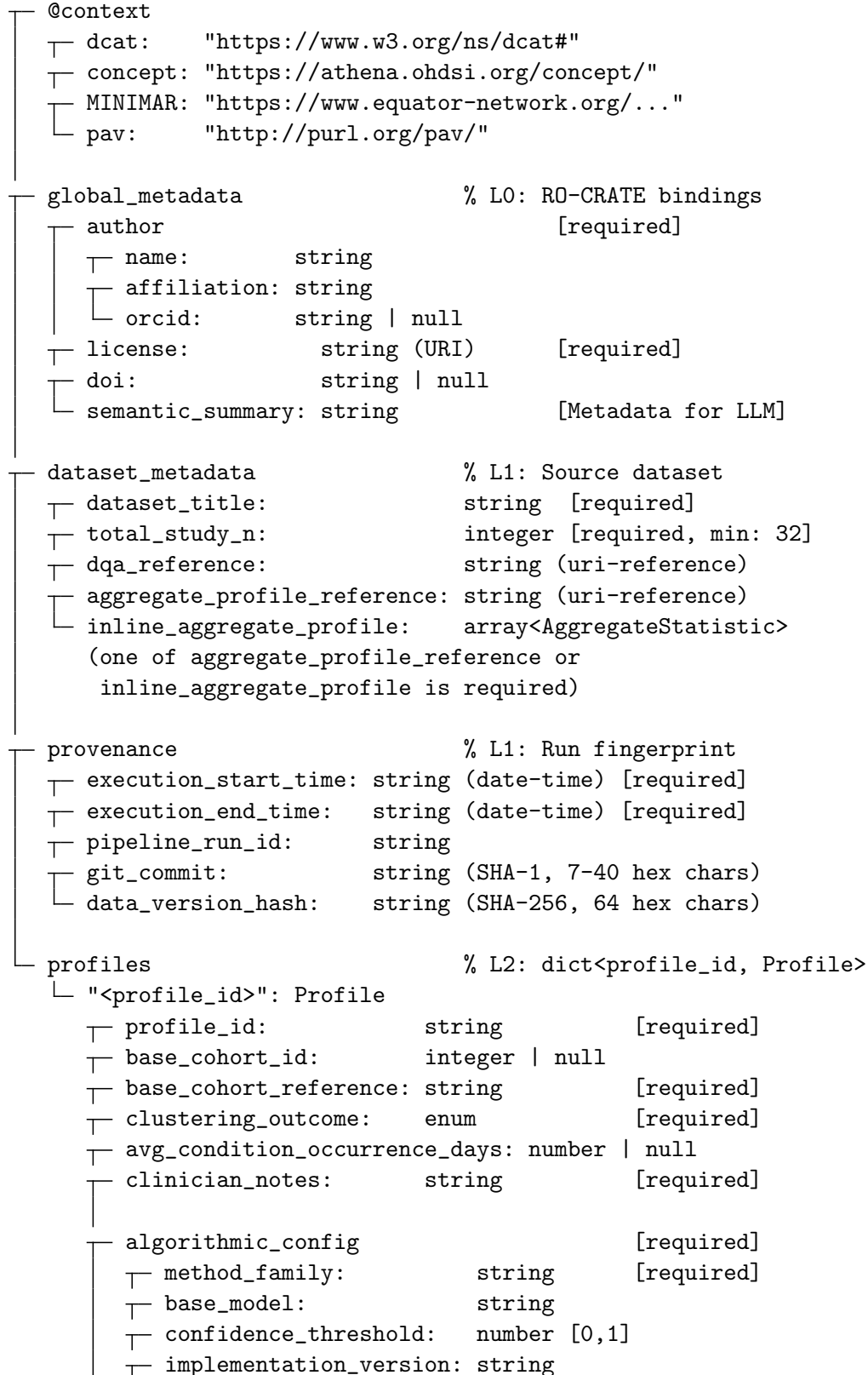
To illustrate the practical flow of the Cluster Phenotype Card, consider the following scenario between two federated research nodes.

1. **The Discovery (Hospital A):** A researcher at Hospital A runs a clustering algorithm on a cohort of Type2 Diabetes patients. The algorithm identifies a high-risk sub-group characterized by HbA1c and elevated BMI. Using the software toolkit, Hospital A generates a `profile.json` card. The pipeline maps features to concept IDs, documents the preprocessing strategy, and computes the SHA-256 integrity hash.
2. **The Transfer:** Hospital A shares the `profile.json` (containing no patient-level data) with Hospital B.
3. **The Validation (Hospital B):** Hospital B receives the card and runs it through their local algorithm for unsupervised clustering. The script applies Hospital A’s preprocessing rules to Hospital B’s local dataset and also has the information for patients matching the dataset used for generating the clusters. The script produces a measurement report: *MCC : 0.560, Silhouette Score: 0.38.* Which can point to a stable cluster.
4. **Generalization:** If several hospitals can find the same cluster with the same feature subset with different datasets and algorithms, the cluster is transport-stable and probably a signal for rule-based phenotype definition. The probabilistic boundaries of the cluster can be safely converted to regular phenotype definitions after generalization.

8 Appendix: JSON Schema

The following tree shows the key/value structure of a valid Profile Card. All top-level blocks are objects (dictionaries). `profiles` and `clusters` are dictionaries keyed by their respective ID strings, not arrays.

PhenoCard (JSON-LD)




```

└ random_seed:          integer | null
└ minimar_reference:    string | null
└ data_preprocessing    [required]
└ ┌ missing_data_strategy: enum [required]
└ └ scaling_method:      enum [required]

└ label_relationship    [required]
└ ┌ assigned_class:      string [required]
└ └ assigned_concept_id: integer (min: 1)

└ clusters              % L3: dict<cluster_id, Cluster>
└ ┌ "<cluster_id>": Cluster
└ └ ┌ cluster_id:        string [required]
└ └ └ phenotype_signature [required]
└ └ └ ┌ integrity_hash:   string (SHA-256)
└ └ └ └ normalization_basis: enum
└ └ └ └ ┌ features: array<VariableConstraint>
└ └ └ └ └ VariableConstraint
└ └ └ └ └ ┌ variable_name:   string [required]
└ └ └ └ └ └ concept_id:      int|null [required]
└ └ └ └ └ └ unit_concept_id: int|null
└ └ └ └ └ └ anchor_operator: enum
└ └ └ └ └ └ anchor_value:    (type per operator)
└ └ └ └ └ └ dist_mean:       number
└ └ └ └ └ └ dist_sd:        number
└ └ └ └ └ └ covariate_importance: number

└ └ └ covariance_matrix: array<CovariancePair>

└ └ └ population          [required]
└ └ └ ┌ n_count:          integer (min: 10) [required]
└ └ └ └ profile_fraction: number [0,1] [required]
└ └ └ └ ┌ study_fraction: number [0,1] [required]

└ └ └ quality_metrics      [required]
└ └ └ ┌ metrics: array<Metric> (minItems: 1)
└ └ └ └ ┌ Metric
└ └ └ └ └ name:          string [required]
└ └ └ └ └ value:         number [required]
└ └ └ └ └ ┌ metric_type: enum
└ └ └ └ └ └ % internal_cluster
└ └ └ └ └ └ % external_confusion
└ └ └ └ └ └ % external_mcc
└ └ └ └ └ └ % class_distribution

```